Toegepaste Informatica

UCLL HOGESCHOOL

Full-Stack Software Development

Spring Security

R. Naudts, J. Pieck, J. Rombouts, B. Van Impe

Back-end security

BREACHES ARE STILL ON THE RISE

- Over 4,100 publicly disclosed data breaches happened last year alone. (or **11 breaches per day**, only on the publicly disclosed data.
- Nearly 109 million accounts were breached in just the 3rd quarter of 2025
- It takes companies an average of **241 days** to identify and contain a breach.
- **60% of breaches** involve a human element like phishing or stolen credentials.
- On average, a data breach costs companies **$4.44 million** (IBM's 2025 Cost of Data Breach Report).

THE FOUR PARTS

User Signup

How to store a password safely

Authentication

“Who are you?"
Verifying a user's identity

Authorization

"What can you do?"
Checking a user's permissions (e.g., ADMIN vs. USER).

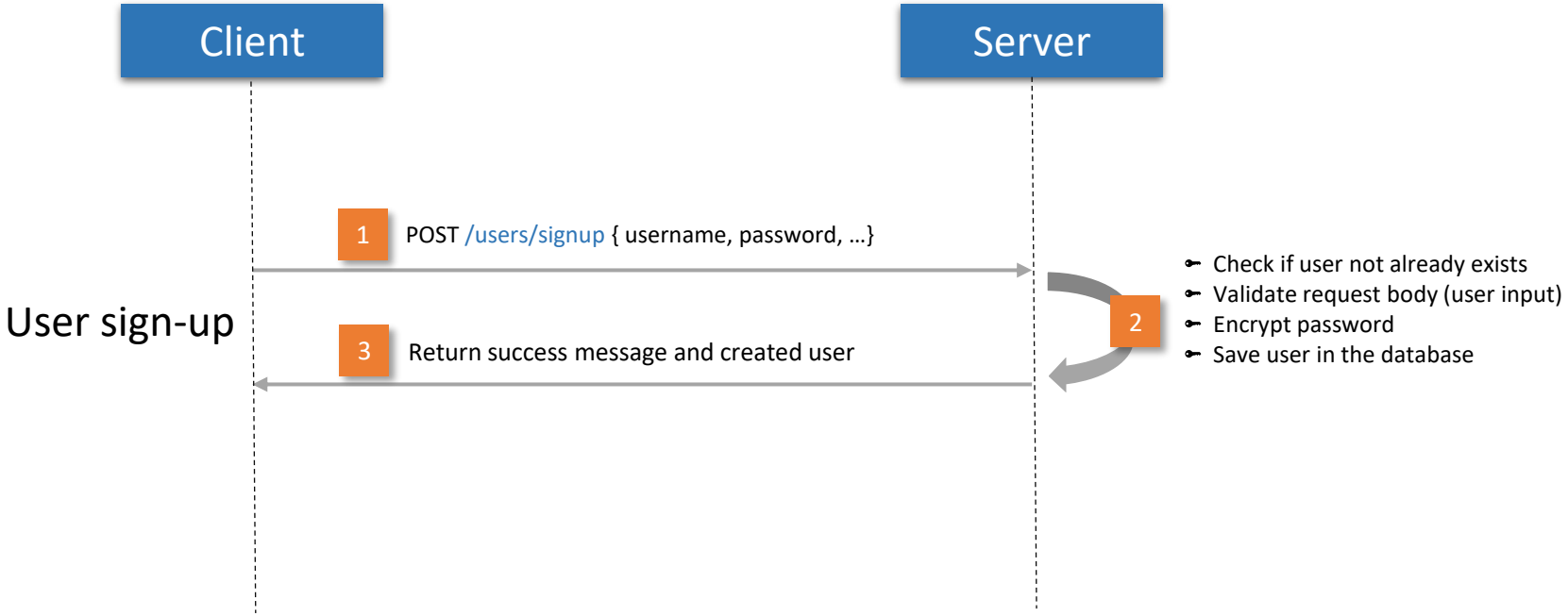
Secure traffic

How to make traffic safer with https

UCLL HOGESCHOOL

Sign up a new user

THE SIGNUP FLOW



STORING PASSWORDS

The #1 Rule: Never Store Passwords in Plain Text

The Problem: Plain Text

Storing "password123" in your database is a critical failure. A single data breach will expose every user's password in clear text.

The Solution: Hashing

We store a one-way-computed "hash" (e.g., $2a$10...). You can't reverse a hash to get the original password, but you can check if a password *produces* the same hash.

ASIDE: HASHING VS ENCRYPTING

Hashing

- Ensures data integrity and verifies authenticity.
- Converts data into a fixed-size string (hash) using a **one-way** function.
- **Irreversible**—cannot retrieve the original data from the hash.
- **Use Cases**: Password storage, digital signatures, checksums.

Encryption

- Protects data confidentiality by making it unreadable to unauthorized users.
- Uses an algorithm and a key to transform data into ciphertext.
- **Reversible**—original data can be retrieved with the correct key.
- **Use Cases**: Secure communication, data storage, file encryption.

Hashing is for verification, encryption is for confidentiality

SETTING THIS UP IN YOUR PROJECT

We need two new dependencies in our pom.xm in order to use the spring boot security feature and configure the application as an OAuth2 Resource Server

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

OAuth 2.0 is the industry-standard protocol for authorization, enabling secure access to resources without sharing credentials.

OAUTH 2 “NOT SHARING CREDENTIALS”

The Valet Key Analogy

Think of your main password as the **Master Key** to your car. It opens the doors, starts the engine, opens the trunk, and unlocks the glovebox.

- **Sharing Credentials:** If you go to a hotel and give the valet your **Master Key**, they can park your car, but they can also open your glovebox and steal your things. You have given them total power.

- **Without Sharing Credentials (OAuth2):** instead of the Master Key, you give the valet a special **Valet Key** (a Token). This key only starts the engine and opens the door. It does not open the trunk or the glovebox.

PASSWORDENCODER

Spring Security (from the starter) provides the PasswordEncoder Interface that is used for hashing a password. We need to provide a @Bean of our preferred implementation (BCrypt).

```
@Configuration
public class SecurityConfig {
    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder( strength: 12);
    }
}
```

- Bcrypt is a password-hashing function. The strength determines how difficult it is to compute the hash. Typically, we use a number between 10 and 14, where higher numbers result in stronger security but slower hashing.

PASSWORDENCODER

In order for a client to create a user we need to provide an endpoint for the signup method on the user controller (taking a DTO as parameter)

```
public record UserInput(
        @NotBlank
        String username,
        @NotBlank
        String password,
        @NotBlank
        String firstName,
        @NotBlank
        String lastName,
        @Email
        String email) {
}
```

```
@PostMapping("/signup")
public User signup(@Valid @RequestBody UserInput userInput) {
    return userService.signup(Role.GUEST, userInput);
}
```

PASSWORDENCODER

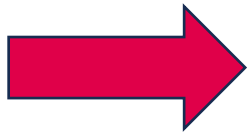
And a signup method in the UserService as well, where we encode the password before storing it

```
public User signup(Role role, UserInput userInput) {
    if (userRepository.existsByUsername(userInput.username())) {
        throw new CoursesException("Username is already in use.");
    }

    final var hashedPassword = passwordEncoder.encode(userInput.password());
    final var user = new User(
            userInput.username(),
            userInput.firstName(),
            userInput.lastName(),
            userInput.email(),
            hashedPassword,
            role
    );

    return userRepository.save(user);
}
```

OUR SEED DATA

- We need to make sure that the passwords of the users we create in dbInitializer are also encoded

```
final var student5User = userRepository.save(new User(
        username: "lindas",
        firstName: "Linda",
        lastName: "Lawson",
        email: "linda.lawson@student.ucll.be",
        password: "lindal123",
        Role.STUDENT
));
```



```
final var student5User = userRepository.save(new User(
        username: "lindas",
        firstName: "Linda",
        lastName: "Lawson",
        email: "linda.lawson@student.ucll.be",
        passwordEncoder.encode( rawPassword: "lindal123"),
        Role.STUDENT
));
```

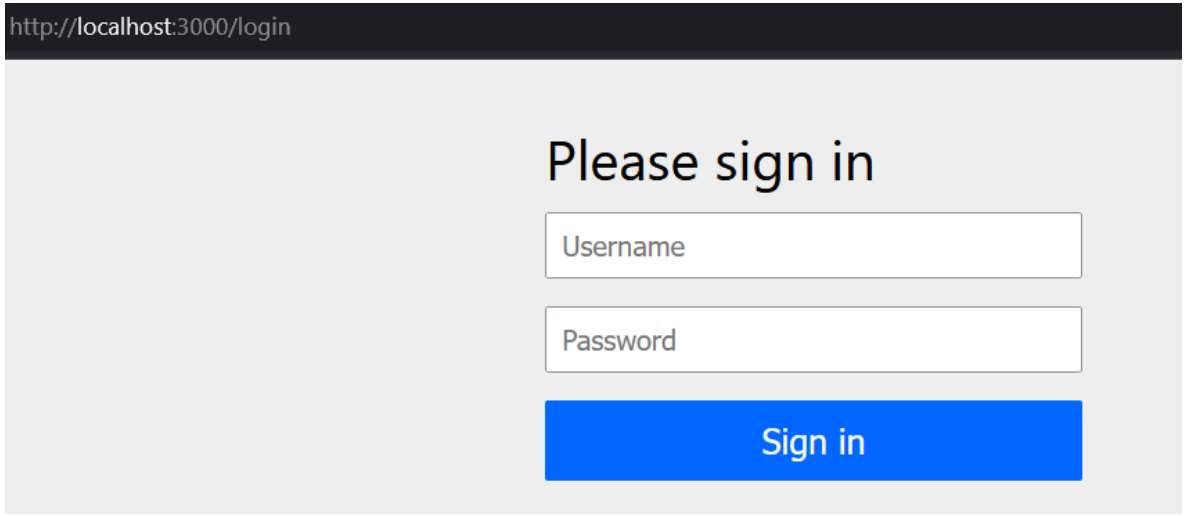
START UP THE APPLICATION

- In the db we can see the hashed passwords:

password text	role text
$2a$12$gFglSWXcE61cIJ0ysE/.yecQ10aiPbFZ6fkkljqHanmJrmalqb...	ADMIN
$2a$12$8wL1YNPZVO3nks7IqK4V7.dZWlLLUqZC3Rg5IdQpdRPsgg...	LECTUR...
$2a$12$1W6uK7igmIvdxe0eqfDKzOu8IXS4ANkPR/8DPTV8FZQvZA...	LECTUR...
$2a$12$hS6ThgygTJ4OrCEzZkllCelvCsrJIubsQ3STyUk9n/kUHAujYC...	LECTUR...
$2a$12$42rG/j5QdakxJzywnGDq2Od50iLIB046CT4qNSTI/cwkLa3Y....	LECTUR...
$2a$12$fygpdumXutF5jl4U460rdejK74eRXh94MDH9iPQRK.bPq5Mv...	STUDENT
$2a$12$e11mR6efaxsHJqTtd0xq/uEkU6TijFrx4X5/vSmQFSeHa0frZ...	STUDENT
$2a$12$2kmt7aIsCxggsYGg4UK5.e2xzEsHnBzbN0JU9naGcoS5irJL...	STUDENT
$2a$12$EKbvdOV07kRz5LQ5wvT1TOfziZ499rIHvuEl2zgvkvf/iQd9cU...	STUDENT
$2a$12$6P8.jlQrsGFfdFbJKzdCmuVdjl03CyZa0jNuXL9Di6SiKqZr6Y...	STUDENT

START UP THE APPLICATION

But any call to localhost:3000 is now redirected to localhost:3000/login and a default login form



By default, all endpoints will be considered to need authentication, so spring boot security will force everyone to log in first, even when trying call the status – or login endpoints or the swagger-ui page.

SECURITYFILTERCHAIN

To define what needs and doesn't need authorization: we can supply a securityFilterChain.

- requestMatchers take urls as parameters
- Order matters! It stops at the first one that matches.

```java
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    return http
            .authorizeHttpRequests(
                    authorizeRequests ->
                            authorizeRequests
                                    // Allow all access to health check
                                    .requestMatchers("/status").permitAll()
                                    // Allow all access to error endpoints
                                    .requestMatchers("/error/**").permitAll()
                                    // Allow all to login and signup
                                    .requestMatchers("/users/login", "/users/signup", "/users/logout").permitAll()
                                    // Allow OpenAPI access
                                    .requestMatchers("/v3/api-docs/**").permitAll()
                                    // Allow Swagger UI
                                    .requestMatchers("/swagger-ui/**", "/swagger-ui.html").permitAll()
                                    .requestMatchers("/test-utils/**").permitAll()
                                    .anyRequest().authenticated())
            .csrf(csrf -> csrf.disable())
            .build();
}
```

Csrf (cross-site request forgery): always disable in a stateless API

OPENAPI

We need to tell our openAPI config about the security, in order to be able to see the status of the endpoint and use the swagger-ui page to test secured end points.

```java
@Configuration
@OpenAPIDefinition(
        security = @SecurityRequirement(name = "bearerAuth")
)
@SecurityScheme(
        name = "bearerAuth",
        type = SecuritySchemeType.HTTP,
        scheme = "bearer",
        bearerFormat = "JWT"
)
public class OpenApiConfig {

    @Bean
    public OpenAPI customOpenAPI() {
        return new OpenAPI()
                .info(new Info()
                        .title("Courses API")
                        .version("1.0")
                        .description("API for managing courses, users, and schedules."));
    }
}
```

SIGN UP A NEW USER

user-controller

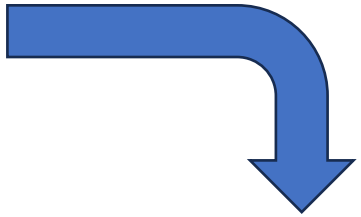
POST `/users/signup`

Parameters

No parameters

Request body required

```
{
  "username": "Test User",
  "password": "Test123",
  "firstName": "Testy",
  "lastName": "McTesterson",
  "email": "test@here.com"
}
```



Query | Query History

```
SELECT * FROM public."user"
where email = 'test@here.com'
```

Data Output | Messages | Notifications

Showing rows: 1 to 1 | Page No

	id [PK] bigint	created_at timestamp w	updated_at timestamp	username text	first_name text	last_name text	email text	password text	role text
1	11	2025-11-...	2025-1...	Test User	Testy	McTesters...	test@here.co...	$2a$12$DRSO...	GUE...

THE FOUR PARTS

User Signup

How to store a password safely

Authentication

“Who are you?"
Verifying a user's identity

Authorization

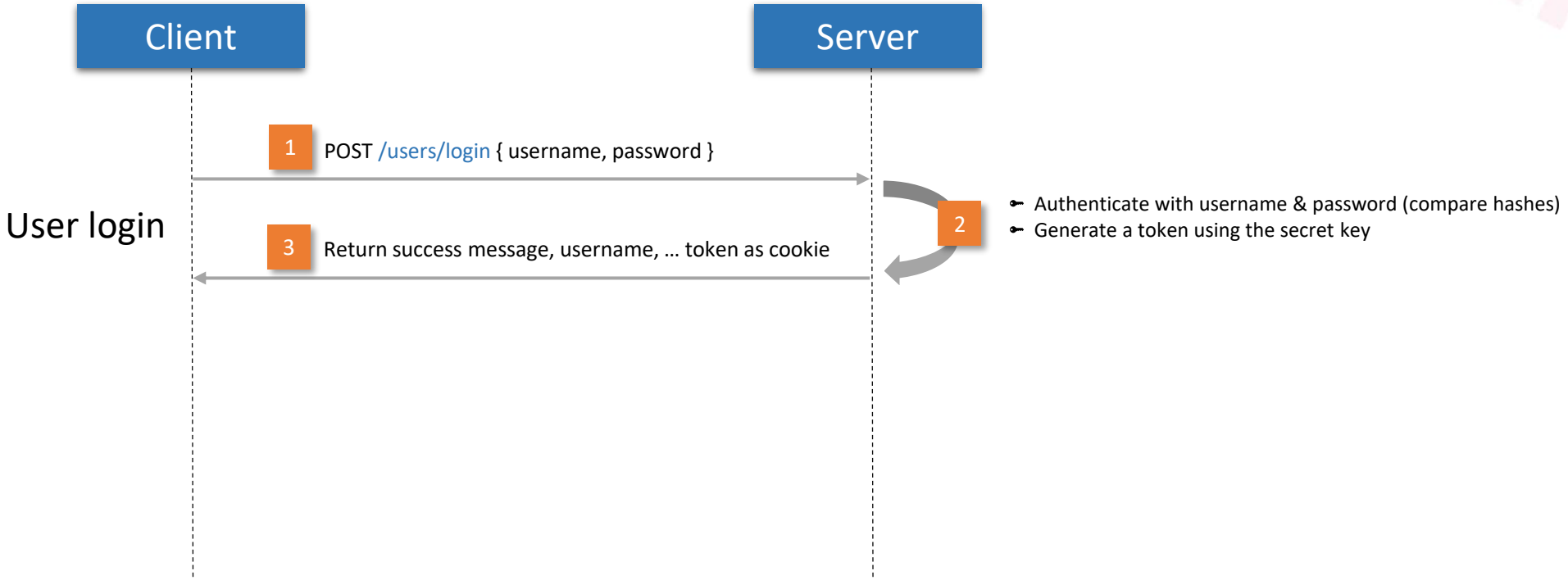
"What can you do?"
Checking a user's permissions (e.g., ADMIN vs. USER).

Secure traffic

How to make traffic safer with https

Authentication

THE AUTHENTICATION FLOW



JWT

JWT = “JSON Web Token”

- Transmit data over a network in a secure way as JSON
- Used for authentication and authorization
- In browser session storage or via secure httpOnly cookies
- Stateless security: The client includes this JWT in the Authorization header of every future request.

BEARER TOKEN

A **Bearer Token** is a type of **access token** used in authentication and authorization processes, most commonly in **HTTP APIs** and web services.

It is called a "bearer" token because **whoever "bears" (possesses) the token is granted access** to the associated resources—no additional proof of identity is required.

A **JWT (JSON Web Token)** is a common format for bearer tokens. It consists of three parts:

- **Header**: Contains metadata (e.g., token type and algorithm).
- **Payload**: Contains **claims** (e.g., user ID, expiration time, roles).
- **Signature**: Ensures the token’s integrity.

RESPONSECOOKIES

ResponseCookie is a class that helps create HTTP cookies

- **Secure by Default**

Encourages secure cookie settings by providing methods to configure security attributes like

- HttpOnly: If true, the cookie is not accessible via JavaScript.
- Secure: If true, the cookie is only sent over HTTPS.
- SameSite: whether the cookie is sent with cross-site requests (Strict, Lax, None).
- maxAge: The maximum age of the cookie in seconds

THE CONTROLLER METHOD

Response is provided by Spring boot, it is the httpResponse for the current request

```java
@PostMapping("/login")
public ResponseEntity<Object> authenticate(@RequestBody AuthenticationRequest authenticationRequest, HttpServletResponse response) {
    var auth = userService.authenticate(authenticationRequest.username(), authenticationRequest.password());

    ResponseCookie cookie = ResponseCookie.from( name: "authToken", auth.token())
            .httpOnly(true)
            .secure(true)
            .path("/")
            .maxAge( maxAgeSeconds: 3600)
            .sameSite(SameSiteCookies.NONE.toString())
            .build();
    response.addHeader(HttpHeaders.SET_COOKIE, cookie.toString());

    // only send token via cookie
    auth = new AuthenticationResponse(auth.message(),  token: null, auth.username(), auth.fullname(), auth.role());
    return ResponseEntity.ok(auth);
}
```

A ResponseCookie is created to store the JWT token.

The cookie is added to the HTTP response's header

The token is removed from the AuthenticationResponse before sending it in the response body (to avoid exposing it directly).

BEARER TOKEN RESOLVER

- A **BearerTokenResolver** is a component responsible for resolving (extracting) the Bearer token from an HTTP request.
- Spring doesn't come with one by default, so we create our own:

```
public class CookieBearerTokenResolver implements BearerTokenResolver {

    private final BearerTokenResolver defaultResolver = new DefaultBearerTokenResolver();

    @Override
    public String resolve(HttpServletRequest request) {
        String tokenFromHeader = defaultResolver.resolve(request);
        if (tokenFromHeader != null) return tokenFromHeader;

        if (request.getCookies() == null) return null;

        return Arrays.stream(request.getCookies()) Stream<Cookie>
                .filter(cookie -> "authToken".equals(cookie.getName()))
                .map(Cookie::getValue) Stream<String>
                .findFirst() Optional<String>
                .orElse( other: null);
    }
```

Filters cookies on the name we gave it, see previous slide

- And add it to our SecurityFilterChain

```
.csrf(csrf -> csrf.disable())
.oauth2ResourceServer(resourceServer ->
        resourceServer.jwt(Customizer.withDefaults())
                .bearerTokenResolver(new CookieBearerTokenResolver()))
.build();
```

SET UP

We need to add some config to the securityConfig:

- set up the Authenticationmanager, which is another interface from Spring Security, responsible for validating user credentials

```
@Bean
public AuthenticationManager authenticationManager(AuthenticationConfiguration authenticationConfiguration)
        throws Exception {
    return authenticationConfiguration.getAuthenticationManager();
}
```

- AuthenticationConfiguration: This is a configuration class provided automatically by Spring Security. It holds the setup details for how authentication should be handled.

SET UP

We need a JwtService Class to generate the tokens

```java
public class JwtService {

    private final JwtProperties jwtProperties;
    private final JwtEncoder jwtEncoder;

    public JwtService(JwtProperties jwtProperties,
                      JwtEncoder jwtEncoder) {
        this.jwtProperties = jwtProperties;
        this.jwtEncoder = jwtEncoder;
    }

    public String generateToken(String username, Role role) {
        final var now = Instant.now();
        final var expiresAt = now.plus(jwtProperties.token().lifetime());
        final var header = JwsHeader.with(MacAlgorithm.HS256).build();
        final var claims = JwtClaimsSet.builder()
                .issuer(jwtProperties.token().issuer())
                .issuedAt(now)
                .expiresAt(expiresAt)
                .subject(username)
                .claim( name: "scope", role.toGrantedAuthority().getAuthority())
                .build();
        return jwtEncoder.encode(JwtEncoderParameters.from(header, claims)).getTokenValue();
    }

    public String generateToken(User user) { return generateToken(user.getUsername(), user.getRole()); }
```

SET UP

The JwtService needs JwtProperties to be able to build a token. Here we add default values for

- the issuer (who created the token)
- lifetime claims (how long is this token valid)

```java
@ConfigurationProperties(prefix = "jwt")
public record JwtProperties(String secretKey,
                            @DefaultValue Token token) {
    public record Token(@DefaultValue("courses_app") String issuer,
                        @DefaultValue("8h") Duration lifetime) {}
}
```

The secret key is read from the application.yaml. For example:

```yaml
jwt:
  secret-key: d2ViNC1ub3Qtc28tc2VjcmV0LWFjY2Vzcy1zZWNyZXQ=
```

Register this record with Spring boot

```java
@Configuration
@EnableConfigurationProperties({JwtProperties.class})
public class SecurityConfig {
    private static final Logger log = LoggerFactory.getLogger(SecurityConfig.class);
```

SET UP THE JWTSERVICE

```java
@Service
@Transactional
public class JwtService {

    private final JwtProperties jwtProperties;
    private final JwtEncoder jwtEncoder;

    public JwtService(JwtProperties jwtProperties,
                      JwtEncoder jwtEncoder) {
        this.jwtProperties = jwtProperties;
        this.jwtEncoder = jwtEncoder;
    }

    public String generateToken(String username, Role role) {
        final var now = Instant.now();
        final var expiresAt = now.plus(jwtProperties.token().lifetime());
        final var header = JwsHeader.with(MacAlgorithm.HS256).build();
        final var claims = JwtClaimsSet.builder()
                .issuer(jwtProperties.token().issuer())
                .issuedAt(now)
                .expiresAt(expiresAt)
                .subject(username)
                .claim( name: "scope", role.toGrantedAuthority().getAuthority())
                .build();
        return jwtEncoder.encode(JwtEncoderParameters.from(header, claims)).getTokenValue();
    }

    public String generateToken(User user) { return generateToken(user.getUsername(), user.getRole()); }
}
```

This service actually generates the token based on username and password or on User.

It uses the values we have set in the previous slide to do so.

SET UP

And it also needs jwtEncoder, which uses the SecretKey to sign a token.
Signing ensures the token's **integrity** and **authenticity**

```java
@Bean
public JwtEncoder jwtEncoder(SecretKey secretKey) {
    // JWK = JSON Web Keys
    final JWK jwk = new OctetSequenceKey.Builder(secretKey)
            .algorithm(JWSAlgorithm.HS256)
            .build();
    // JWK set
    final var jwks = new ImmutableJWKSet<>(new JWKSet(jwk));
    return new NimbusJwtEncoder(jwks);
}

@Bean
public SecretKey secretKey(JwtProperties jwtProperties) throws NoSuchAlgorithmException {
    final var secretKeyProperty = jwtProperties.secretKey();
    if (secretKeyProperty == null || secretKeyProperty.isEmpty()) {
        log.warn("No secret key configured, generating a random key");
        final var secretKeyGenerator = KeyGenerator.getInstance( algorithm: "AES");
        return secretKeyGenerator.generateKey();
    } else {
        final var bytes = Base64.getDecoder().decode(jwtProperties.secretKey());
        // Even though we're doing HMAC-SHA256, not AES, we still need to provide an algorithm
        return new SecretKeySpec(bytes,  algorithm: "AES");
    }
}
```

SET UP

Since Spring Security works with the interface UserDetail, we need to implement it

```java
public record UserDetailsImpl(User user) implements UserDetails {

    @Override
    public String getUsername() { return user.getUsername(); }

    @Override
    public String getPassword() { return user.getPassword(); }

    @Override
    public Collection<? extends GrantedAuthority> getAuthorities() {
        return List.of(user.getRole().toGrantedAuthority());
    }
}
```

What Are Granted Authorities?

- **Roles**: High-level permissions (e.g., ROLE_ADMIN, ROLE_USER).
- **Authorities**: Fine-grained permissions (e.g., READ_PRIVILEGE, WRITE_PRIVILEGE).

In Spring Security, both roles and authorities are represented as GrantedAuthority objects.

SET UP

Since Spring Security also works with the interfaces UserDetailsService and UserDetailsPasswordService, we need to implement it

```java
@Service
@Transactional
public class UserDetailsServiceImpl implements UserDetailsService, UserDetailsPasswordService {

    private final UserRepository userRepository;

    public UserDetailsServiceImpl(UserRepository userRepository) { this.userRepository = userRepository; }

    @Override
    public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
        return new UserDetailsImpl(userRepository.findByUsername(username).orElseThrow(() -> new UsernameNotFoundExceptior
    }

    @Override
    public UserDetails updatePassword(UserDetails userDetails, String newPassword) {
        final var user = ((UserDetailsImpl) userDetails).user();
        user.setPassword(newPassword);
        return userDetails;
    }
```

SET UP

Role needs to be expanded to return a GrantedAuthority.

A **GrantedAuthority** represents a **permission** granted to an authenticated user

Since we will be using the role of a user to determine their permission, this code maps the enum value to a GrantedAuthority

```
public enum Role {

    ADMIN,
    STUDENT,
    LECTURER,
    GUEST;

    public GrantedAuthority toGrantedAuthority() {
        return new SimpleGrantedAuthority( role: "ROLE_" + name());
    }

    @Override
    @JsonValue
    public String toString() { return super.toString().toLowerCase(Locale.ROOT); }
}
```

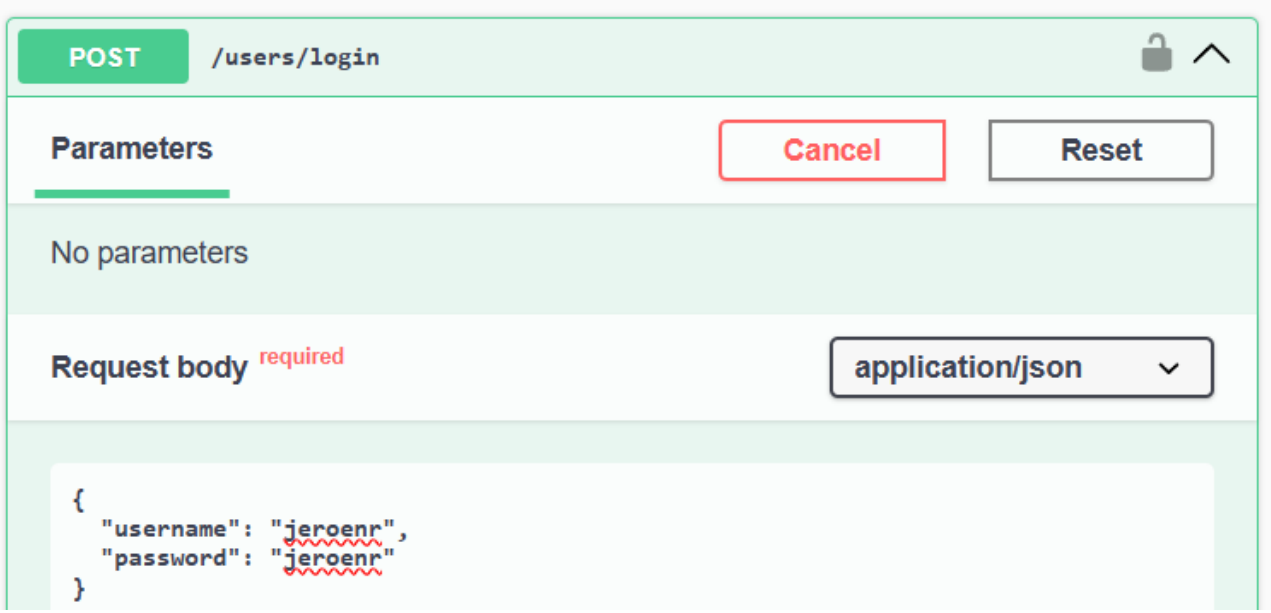
SET UP: ALMOST THERE

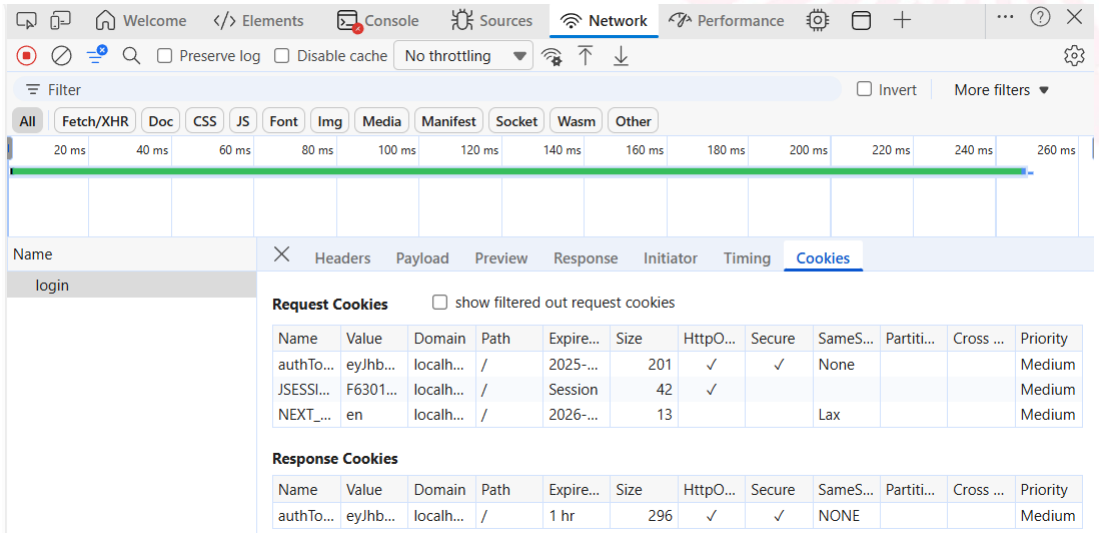
Finally, we can write the userService method

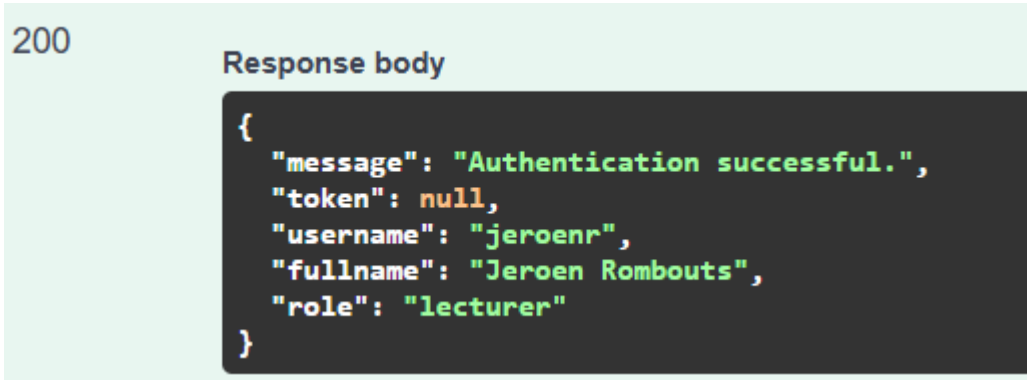
```
public AuthenticationResponse authenticate(String username, String password) {
    final var usernamePasswordAuthentication = new UsernamePasswordAuthenticationToken(username, password);
    final var authentication = authenticationManager.authenticate(usernamePasswordAuthentication);
    final var user = ((UserDetailsImpl) authentication.getPrincipal()).user();
    final var token = jwtService.generateToken(user);
    return new AuthenticationResponse(
            message: "Authentication successful.",
            token,
            user.getUsername(),
            user.getFullName(),
            user.getRole()
    );
```

the **principal** represents the currently authenticated user. It is typically a UserDetails object, we cast it to our implementation of UserDetail

TESTING A LOG-IN







We can authenticate via Swagger. The token is not present in the response, but it is present in the cookie

THE FOUR PARTS

User Signup

How to store a password safely

Authentication

“Who are you?"
Verifying a user's identity

Authorization

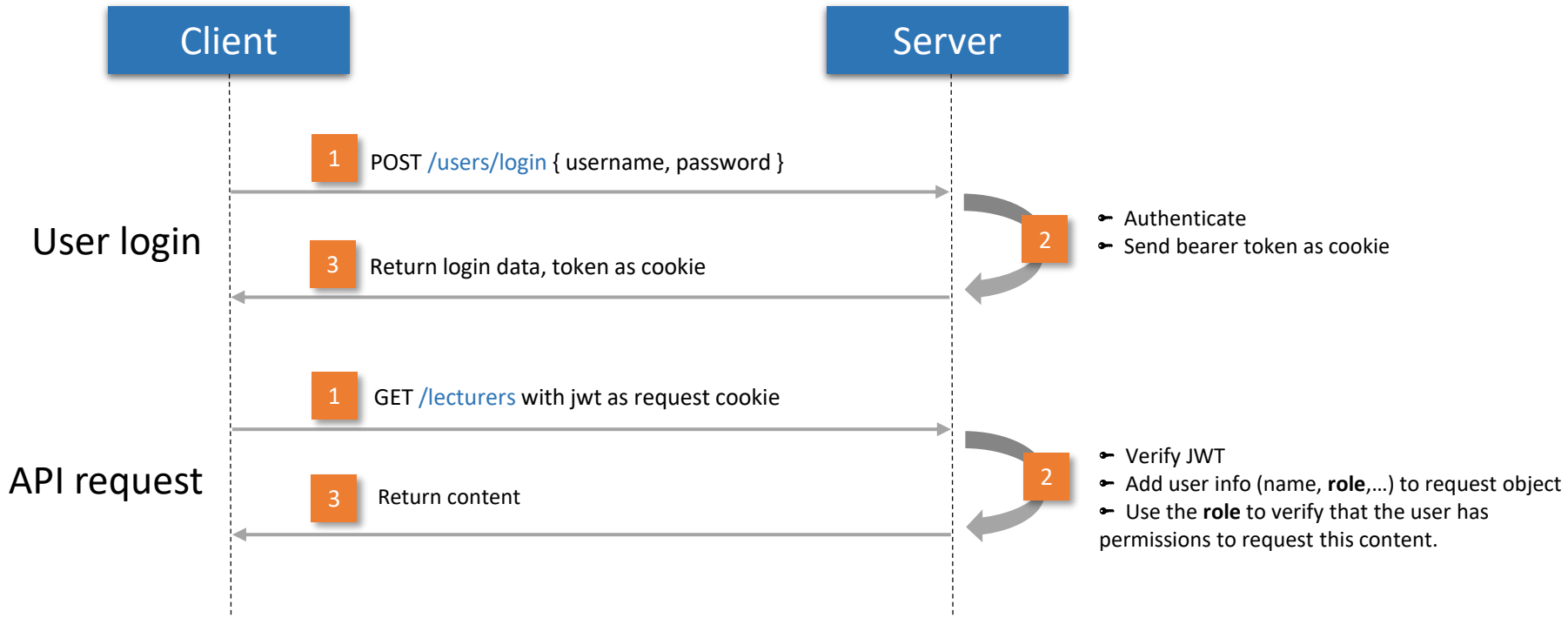
"What can you do?"
Checking a user's permissions (e.g., ADMIN vs. USER).

Secure traffic

How to make traffic safer with https

Authorization

AUTHORIZATION FLOW



AUTHORIZATION

The Goal:

- Only a user with the role admin can add students.

SET UP (AGAIN)

In SecurityConfig:

- We need to provide a JwtDecoder to parse and validate the structure and signature of a JWT and read the contents.

```
@Bean
public JwtDecoder jwtDecoder(SecretKey secretKey) {
    return NimbusJwtDecoder.withSecretKey(secretKey).macAlgorithm(MacAlgorithm.HS256).build();
}
```

- To allow security on method level, we need to add the @EnableMethodSecurity to our SecurityConfig class:

```
@Configuration
@EnableMethodSecurity
@EnableConfigurationProperties({JwtProperties.class})
public class SecurityConfig {
```

ADD AUTHORIZE

@PreAuthorize: Checks authorization before a method is invoked. If the condition is not met, the method is not executed, and an AccessDeniedException is thrown.

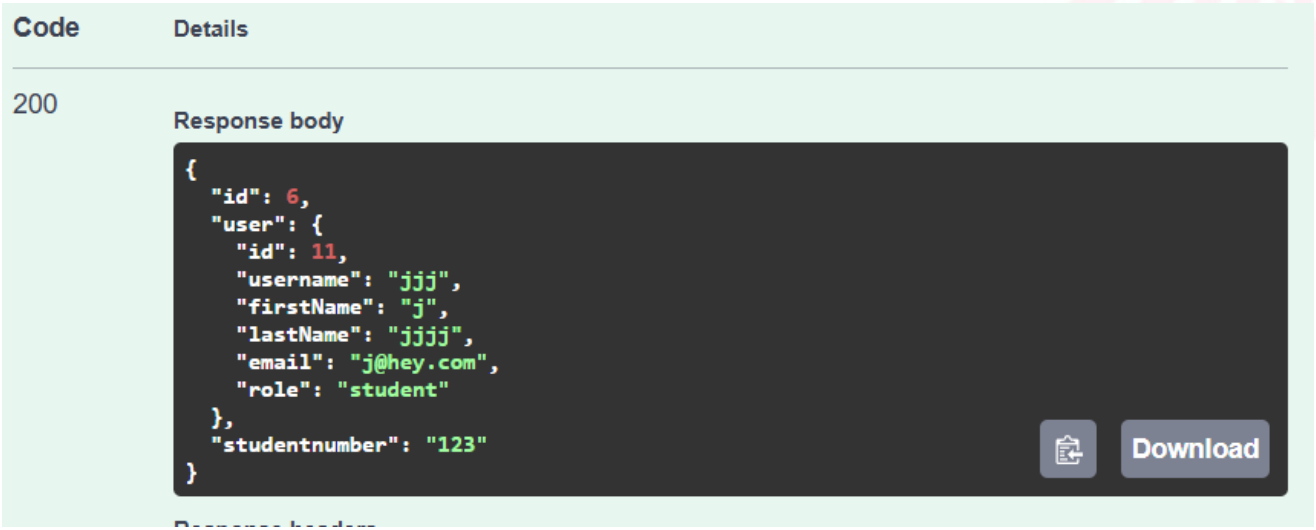
@PostAuthorize: Checks authorization after a method is invoked but before the result is returned. If the condition is not met, a security exception is thrown, and the result is not returned. This is often used to filter out sensitive information.

For our use case the @PreAuthorize is the most useful one:

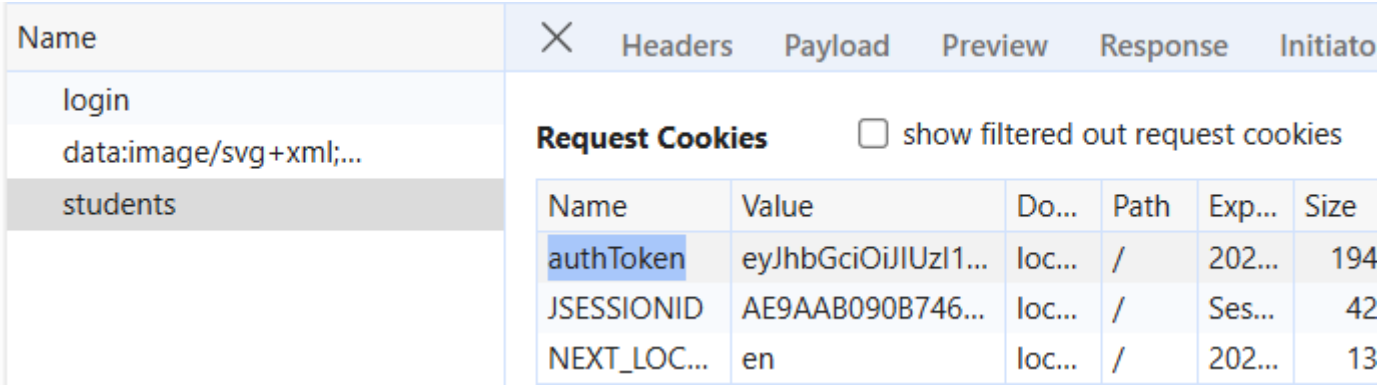
```
@PostMapping
@PreAuthorize("hasRole('ADMIN')")
public Student addStudent(@RequestBody @Valid StudentInput studentInput) {
```

TEST IT OUT

- Log in as admin in Swagger
- Try to add a student:



- In the console you can see that our authToken has been added to the request headers



THE FOUR PARTS

User Signup

How to store a password safely

Authentication

“Who are you?"
Verifying a user's identity

Authorization

"What can you do?"
Checking a user's permissions (e.g., ADMIN vs. USER).

Secure traffic

How to make traffic safer with https

Later, time permitting

Testing

ADD DEPENDENCY

- <dependency>
  <groupId>org.springframework.security</groupId>
  <artifactId>spring-security-test</artifactId>
  <scope>test</scope>
  </dependency>

- This dependency will give access to @WithMockUser

SERVICE TEST

- @WithMockUser simulates a logged in user:

```
@Test
@WithMockUser(username = "josbosmans", roles = {"LECTURER"})
public void givenUserIsAuthenticatedAsLecturer_whenGetSchedule_thenScheduleIsReturned() {
    final var schedule = Mockito.mock(Schedule.class);

    Mockito.when(scheduleRepository.findByLecturer_User_Username("josbosmans")).thenReturn(List.of(schedule))

    final var authentication = SecurityContextHolder.getContext().getAuthentication();
    final var list = scheduleService.getSchedules(authentication);
    Assertions.assertEquals( expected: 1, list.size());
    Assertions.assertEquals(schedule, list.getFirst());
}
```

E2E TESTS

- Generate the token yourself by using the jwtService

```
@Test
public void givenUserIsAuthenticatedAsLecturer_whenGetSchedule_thenScheduleIsReturned() {
    final var token = jwtService.generateToken( username: "bramb", Role.LECTURER);

    client
            .get() RequestHeadersUriSpec<capture of ?>
            .uri( uri: "/schedules") capture of ?
            .header( headerName: "Authorization",  ...headerValues: "Bearer " + token)
            .exchange() ResponseSpec
```